

Decision models, not chat models

Email triage with TypeSafe Jev

Code, data, and every run in this deck: github.com/skiingfalcon/jev-email-cascade

Why this exists

Inbox volume is **routing**, not drafting.

A chat model can do it.

Slow, expensive, and the confidence it types is not a probability.

The bet

Jev decides. Ordinary code routes. A generative model only handles leftovers.

Three jobs for a transformer

	Role	Examples
Writers	next token	GPT, Claude, Llama
Readers	whole sequence → vector	BERT, embeddings
Deciders	whole sequence → judgments	Jev

Same family of model. Different last layer.

Same math

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T + M}{\sqrt{d_k}} \right) V$$

M is the mask. The **head** is what you read out.

At scale, QK^T is a huge matrix multiply. That is why this runs on **GPUs**.

How they differ

- **Writer** — causal mask. Guess the next word.
- **Reader** — open mask. Emit an embedding.
- **Decider** — open mask. Emit Choice / Score / Noul.

A decider is an encoder used for a decision, not for search.

The KV cache

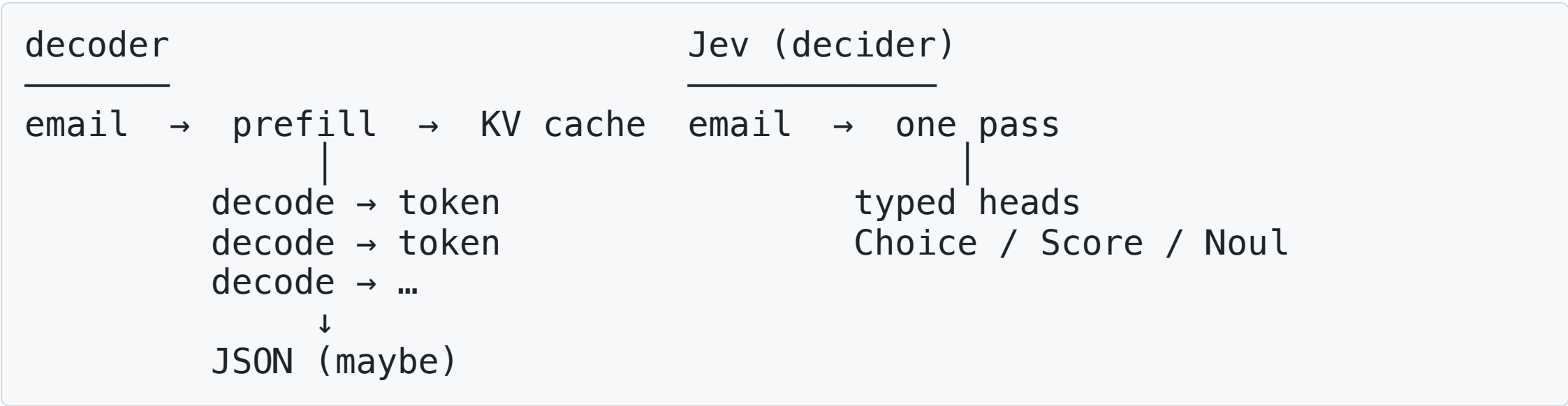
Attention needs a **key** and a **value** for every token already seen.

```
prompt — prefill (one pass) → write K,V into cache
                                     |
token   — decode (one step) → read cache, append this token's K,V
token   — decode —————→ read ...append
token   — decode —————→ read ...append
```

Without the cache, every decode step recomputes K,V for the whole prefix.

A **reader / decider** has no next token: one pass, discard. Nothing to keep.

Decoder vs Jev



	Decoder	Jev
Steps	1 + <i>every output token</i>	1
KV cache	grows while it writes	computed once, never kept
Output	words	probabilities
Implementation	prefill + decode loop	forward pass + post-processing

This project

74 synthetic labelled emails

8 typed questions

One shared answer schema

Score against gold labels

What we send Jev

State — prepared email
(from, subject, date, body)

Questions — not a prompt

- 1 Choice
- 1 Score
- 6 Noul

One forward pass. No prose back.

Three output types

Choice — one option + distribution + confidence

Score — position on a rubric (can be fractional)

Noul — $P(\text{yes})$ in $[0, 1]$. No separate confidence.

Choice — category

support · sales · billing · hr
vendor · internal · spam · **other**

Returns the winner, the full distribution, and a confidence.

Score — **priority**

0	no action
1	reply wanted, nothing at risk
2	act in a few days
3	act within ~2 days

Expected index + distribution across levels.

Noul — six flags

Flag	Question
<code>awaiting_reply</code>	asks us to do something?
<code>deadline_present</code>	a real date/time?
<code>dissatisfied</code>	complaint?
<code>needs_decision</code>	judgment / money / approval?
<code>opportunity</code>	revenue for <i>us</i> ?
<code>injection_suspected</code>	talking to the classifier?

What **auto** / **review** mean

auto — stop. No human, no LLM.

review — queue a person, with the evidence attached.

llm — optional draft, still not the decision.

One email in

Trouble connecting our account... I'm frustrated... Could you help as soon as possible?

Jev out

category	support · conf 1.0
priority	2.01 · conf 0.99
awaiting_reply	0.98
deadline_present	0.03
dissatisfied	0.96
needs_decision	0.18
opportunity	0.16
injection_suspected	0.02

Then **code** says: `route = review`

(reason: dissatisfied)

Results — 74 emails

	Jev	gpt-5.6-terra	GLiNER2.5*
Category	95.9%	97.3%	45.9%
Priority exact	78.4%	90.5%	16.2%
Flags (mean)	89.0%	90.5%	45.5%
Auto	4	26	1
Cost	\$0.004	\$0.26	\$0
Latency p50	272 ms	1.6 s	79 ms

Jev vs Terra: accuracy close, cost and speed not.

* first wiring — see "The free one".

The number that matters

When they were **wrong** on category:

- Jev said 0.39 – 0.65 → **review**
- Terra said 0.93 – 0.96 → would have **auto-filed**

Jev's 4 autos were clean.

Terra's 26 autos: 8 had a wrong flag.

On the six flags, both were confidently wrong sometimes:

Jev 21, Terra 29 (of 444) — mostly on labels we'd argue about too.

Where Jev was wrong

- "within 24 hours" / "within 48 hours" → `deadline_present` 0.25–0.35
- "stand-up moved to 10am" → `deadline_present` 0.86–0.89, priority ~1.7 (true: 0)
- 12K-char padded email → category `other` 0.39, deadline 0.29 — both missed

The free one

GLiNER2.5 — open-weight encoder, same class as Jev.

Typed labels in, scores out, one forward pass.

79 ms. \$0. Scored at chance on yes/no.

Every flag sat in the "cannot tell" band on 50–59 of 74 emails.

What we would change to auto more

Not a new model.

1. Stop auto-escalating `needs_decision`
2. Tighten that question's wording — then `deadline_present`
3. Then retune thresholds

Takeaway

Jev is a **decider**.

Python is the **router**.

Chat earns its cost only on leftovers.